APPROACH NOTE

Problem 1: The market data lies to us

Flent Forward Deployed Engineer take-home | Bangalore rental listings

The problem, as I see it

A bad median looks just as precise as a good one. That is the real failure here. In the case packet, a median across all 86 scraped listings is ₹59,000. It looks clean on a dashboard. But that pull also contains an obvious typo at ₹12,000, an outlier at ₹1,85,000, old listings, broker copies, and the subject home itself posted under the wrong BHK. A number can be neat while the evidence behind it is not.

The person who feels this is the acquisition owner. They are deciding whether to put a multi-year rent obligation and roughly ₹2.6 lakh of capital behind one home. Supply is also negotiating against the benchmark. If the comparable set is really four homes dressed up as 30, one bad listing can move the answer by thousands. The brief makes the cost concrete: roughly ten extra vacancy days can push breakeven back by a month.

The framing I used

I did not treat this as a data-cleaning exercise. The problem is silent judgment. A system can silently trust a row, or silently delete it, and both choices turn an opinion into arithmetic. My aim is to make those choices visible. Each listing keeps its original values, its grade, and a plain reason for what happened to it. The market estimate then carries the confidence it has actually earned.

That changes the product from “here is the market rent” to “here is the evidence we have, what it supports, and what is still uncertain.” It is a small distinction in wording, but it changes how safely the team can act.

The claim I am willing to make

For this deal, the layer can recover a defensible asking-rent benchmark from a polluted pull without hiding the messy parts. It should not pretend to predict the rent a tenant will finally sign for. That needs signed-rent history, which this packet does not contain.

How the trust layer works

The flow is simple on purpose. Parse the raw file, normalize dates and names, fold obvious broker cross-posts into one physical unit, merge society spellings only when the evidence proves they refer to the same place, grade each row, and calculate a weighted median with a confidence level. Every threshold sits in one config object and is shown in the review surface.

What stays deterministic

Anything that moves the benchmark stays in plain code: parsing, matching rules, weights, median calculation, confidence checks and the final comparison. The same file should give the same result later. That matters when a reviewer asks why a listing was excluded or when a decision needs to be replayed after the outcome is known.

Where a person should decide

I deliberately leave two decisions open for review. First, cross-society near-matches go to a suspect queue rather than being auto-merged. CP-0026 and CP-0053 have similar rents, areas and dates, but their deposits and society names disagree. They may be similar homes, not the same home. Second, a reviewer can exclude or reinstate any row with a required reason. The estimate recomputes and the override sits beside the machine's original reasoning.

Where AI helps

AI is useful upstream: reading messy listing text, proposing normalized fields, and surfacing possible name matches for a person to inspect. I would not put a model inside the final calculation. A fluent explanation that misquotes one rupee is worse than no explanation. The decision math should remain inspectable and deterministic; AI can help prepare evidence, not quietly decide it.

How the layer handles imperfect evidence

Evidence condition	How I treat it
Missing	Keep it blank. A missing area weakens a match; it never becomes a made-up number.
Stale	More than 30 days dark is excluded. Fifteen to 30 days is kept at half weight.
Conflicting	Show the spread inside duplicate clusters and send cross-society near-matches to review.
Unreliable	Quarantine or downweight with a written reason. Keep the original row in the audit trail.

What the proof shows

The riskiest assumption was that understandable rules would leave enough good evidence to act on. The end-to-end run supports that claim for this case. The byte-checked CSV goes in unchanged. Twenty-one of the 86 listings contribute to the same-society, 2BHK, semi-furnished comparison set. Their effective sample is 16.0 after downweighting weaker evidence. The weighted-median asking-rent benchmark is ₹59,500, with a ₹58,500 to ₹60,000 bootstrap band and a ₹55,500 to ₹63,000 observed range.

The result is HIGH confidence under the defined ladder: the evidence comes from four platforms, has a median age of six days, and the leave-one-out swing is ₹0. Removing any one contributor does not change the median. The raw ₹59,000 result was close, but only by accident. In this packet the bad rows happen to offset one another. Cleaning still bought a known sample, a usable range, and evidence that is stable enough to challenge.

The trap that mattered most

CP-0081 is the subject flat cross-posted by a broker with a 3BHK label. It shares the subject's 1,175 sqft area, ₹2.8 lakh deposit and recent posting date. The layer quarantines it as a subject echo. If it were allowed into the comps, the landlord's own ask would help validate itself. It also explains why the 3BHK segment correctly returns N=0 instead of pretending a mislabeled row is evidence.

A result can be useful and still be incomplete

The landlord asks ₹56,000 plus ₹5,000 maintenance. Listing rents do not reliably say whether maintenance is included, so the honest verdict is two readings: the all-in ask is 2.5% above the benchmark; the base rent is 5.9% below it. I would not pick the reading that makes the deal look better. The sign flip is the finding, and it identifies the next piece of data to collect.

The deliberate failure case and scope cuts

When I ask for a furnished benchmark, only four rows survive, with an effective sample of 3.0. The system labels that LOW confidence and gives a collection list rather than a recommendation. For 3BHK it says INSUFFICIENT. This is intentional: refusing a weak answer is part of the product.

- I did not build an achieved-rent model. Every listing is an ask; signed rents are needed before it can be calibrated.
- I did not impute maintenance, adjust for floor or view, or add synthetic rows. At this sample size, those would create invented precision.
- I did not build a locality map. The packet has anonymised societies and no coordinates; the real signal here is behavioural, not geographic.
- I kept probabilistic record linkage as the scale path, not the v1 default. Strict matching plus a human queue is easier to audit at 86 rows.

What I would validate next

First: capture whether maintenance is included in each listing rent. It changes this deal's verdict from above market to below market. I would test the field on ten live deals by calling the surviving fresh comps. If at least 80% can state inclusion clearly, we should see the two-reading verdict collapse into one. That is stronger evidence than choosing a default today.

Second, I would log the benchmark beside eventual signed rent and days-to-fill. A fast delisting may hint that a listing cleared near its ask, but it can also be an expiry or repost, so it remains display-only for now. After enough signed outcomes, the team can measure the gap between asking rents and achieved rents instead of assuming they are the same.

Assumptions I made visible

Choice	Working rule	Why I chose it
Staleness	>30 days out; 15-30 days at 0.5 weight	A half-weight middle ground is safer than a sharp cut-off.
Duplicates	Dates within 3 days; area within 25 sqft; rent or deposit agrees	Strict rules protect against merging different homes that merely look alike.
Aliases	Two independent cross-posted homes bridge name spellings	One bridge can itself be a bad listing.
Confidence	Effective sample, source count, recency and leave-one-out swing	A stable number from very thin evidence should still be low confidence.

These are working assumptions, not Flent policy. They are deliberately round, visible and easy to change. The release should show what changes when a threshold changes, rather than make the constants disappear into the pipeline.

How I would take this from proof to use

1. Start with canonical ingest and the tested rules engine, so every raw field and decision can be replayed.
2. Add deduplication and alias review with a precision check on sampled clusters before allowing automatic folding at scale.
3. Ship the benchmark, band and confidence ladder inside the BOSS deal page, with overrides and reasons visible in two clicks.
4. Run in shadow mode on ten live deals. Watch reviewer override rate and the share of LOW or INSUFFICIENT outputs before widening rollout.

How I used AI

I used Claude to explore the case packet, surface traps and pair-build the tested pipeline. I checked results through the tests, golden run and independent calculations. I made the choices on framing, thresholds, human review and scope. The repository includes the full AI usage log.